6:54

ukka-venugopalam.github.io

Interactive Graph Algorithm Visualizer

Run | Edit | Delete | Clear Canvas

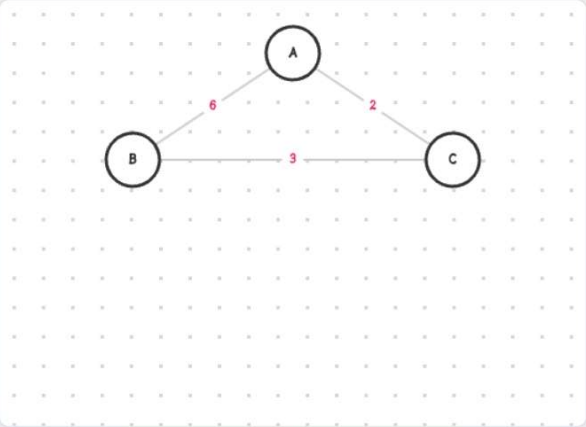
Start Node: A Speed:

BFS | DFS | Dijkstra | Floyd-Warshall | Prim's (MST) | Kruskal's (MST)

Reset View

Data Structure Status

Ready to start



Unvisited | In Memory | Processing | Visited

Algorithm Concept

Select an algorithm to see how it explores the graph differently!

```
Time: O(V + E)
Space: O(V)
```

Execution Log

[redacted]

6:55

ukka-venugopalam.github.io

Interactive Graph Algorithm Visualizer

Run | Edit | Delete | Clear Canvas

Start Node: A Speed:

BFS | DFS | Dijkstra | Floyd-Warshall | Prim's (MST) | Kruskal's (MST)

Reset View

Breadth-First Search

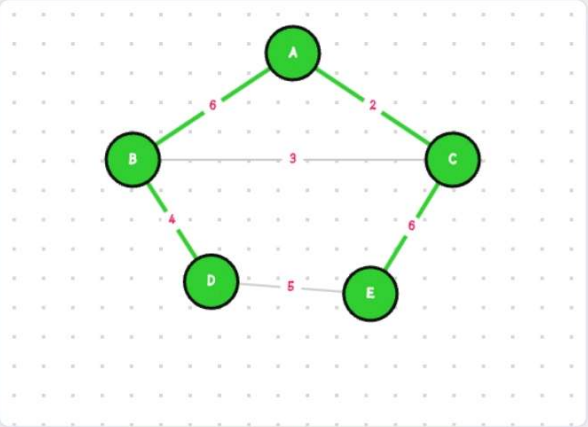
Explores layer by layer. Uses Queue (FIFO).

The green lines show the BFS Tree.

```
Time: O(V + E)
Space: O(V)
```

Queue (FIFO)

Empty



Unvisited | In Memory | Processing | Visited

Execution Log

```
[18:55:22] Start: Pushing Node A to Queue.
[18:55:28] Finished: BFS path complete!
```

BFS Traversal Route

A → B → C → D → E

6:55 Vo) LTE 0.50 KB/s 4G+ 48%

ukka-venugopalam.github.io 13

Interactive Graph Algorithm Visualizer

Run | Edit | Delete | Clear Canvas

Start Node: A Speed:

BFS | DFS | Dijkstra | Floyd-Warshall | Prim's (MST) | Kruskal's (MST)

Reset View

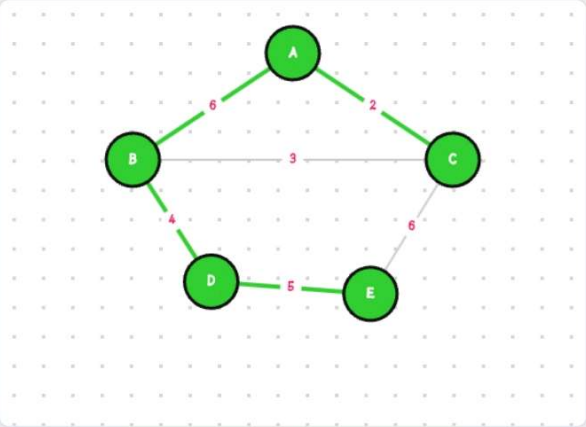
Depth-First Search

Plunges as deep as possible. Uses Stack (LIFO). The green lines show the DFS Tree.

```
Time: O(V + E)
Space: O(V)
```

Stack (LIFO)

Empty



Unvisited | In Memory | Processing | Visited

Execution Log

```
[18:55:33] Start: Pushing Node A to Stack.
[18:55:39] Finished: DFS path complete!
```

DFS Traversal Route

A → B → D → E → C

6:55 Vo) LTE 2.60 KB/s 4G+ 48%

ukka-venugopalam.github.io 13

Interactive Graph Algorithm Visualizer

Run | Edit | Delete | Clear Canvas

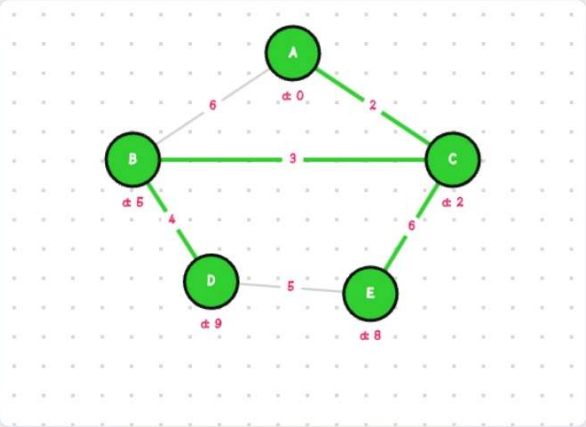
Start Node: A Speed:

BFS | DFS | Dijkstra | Floyd-Warshall | Prim's (MST) | Kruskal's (MST)

Reset View

Priority Queue (Min Dist)

Empty



Unvisited | In Memory | Processing | Visited

Dijkstra's Algorithm

Finds shortest paths from start node. Uses Priority Queue.

Time: O(V + E)
Space: O(V)

Execution Log

[18:55:43] **Start:** Setting Node A distance to 0.

[18:55:44] ↳ **Update:** Shorter path to B is 6!

[18:55:44] ↳ **Update:** Shorter path to C is 2!

[18:55:45] ↳ **Update:** Shorter path to B is 5!

[18:55:46] ↳ **Update:** Shorter path to E is 8!

[18:55:47] ↳ **Update:** Shorter path to D is 9!

[18:55:50] **Finished:** Shortest paths found!

Shortest Distances from A

Node A: 0 | Node B: 5 | Node C: 2 | Node D: 9 | Node E: 8 | Node F: ∞

6:56 Vo) LTE 0.00 KB/s 4G+ 48%

ukka-venugopalam.github.io 13

Interactive Graph Algorithm Visualizer

Run | Edit | Delete | Clear Canvas

Start Node: A Speed:

BFS | DFS | Dijkstra | Floyd-Warshall | Prim's (MST) | Kruskal's (MST)

Reset View

Floyd-Warshall

Finds all-pairs shortest paths via DP Matrix updating.

Time: O(V + E)
Space: O(V)

Execution Log

C is shorter! (9)

[18:56:01] ↳ **Update:** D → A via C is shorter! (9)

[18:56:01] ↳ **Update:** E → A via C is shorter! (8)

[18:56:02] ↳ **Update:** E → B via C is shorter! (9)

[18:56:02] **Phase 4/6:** Testing all paths going *via* Node D.

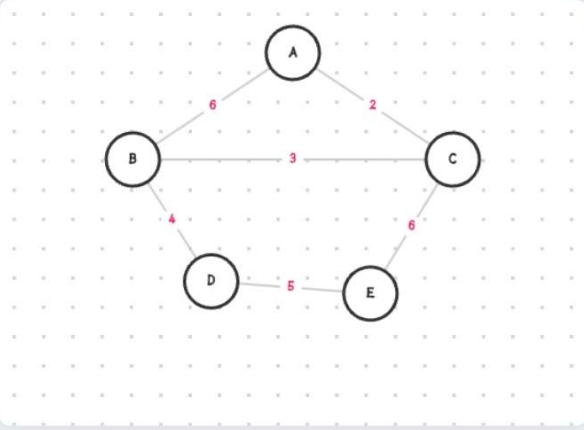
[18:56:03] **Phase 5/6:** Testing all paths going *via* Node E.

[18:56:03] **Phase 6/6:** Testing all paths going *via* Node F.

[18:56:03] **Finished:** All-Pairs Shortest Paths computed!

Priority Queue (Min Dist)

	A	B	C	D	E	F
A	0	5	2	9	8	∞
B	5	0	3	4	9	∞
C	2	3	0	7	6	∞
D	9	4	7	0	5	∞
E	8	9	6	5	0	∞
F	∞	∞	∞	∞	∞	0



Default | Source (i) | Dest (j) | Via (k)

Floyd-Warshall Final Matrix

	A	B	C	D	E	F
A	0	5	2	9	8	∞
B	5	0	3	4	9	∞
C	2	3	0	7	6	∞
D	9	4	7	0	5	∞
E	8	9	6	5	0	∞
F	∞	∞	∞	∞	∞	0

6:56 Vo) LTE 0.00 KB/s 4G+ 48%

ukka-venugopalam.github.io 13

Interactive Graph Algorithm Visualizer

Run | Edit | Delete | Clear Canvas

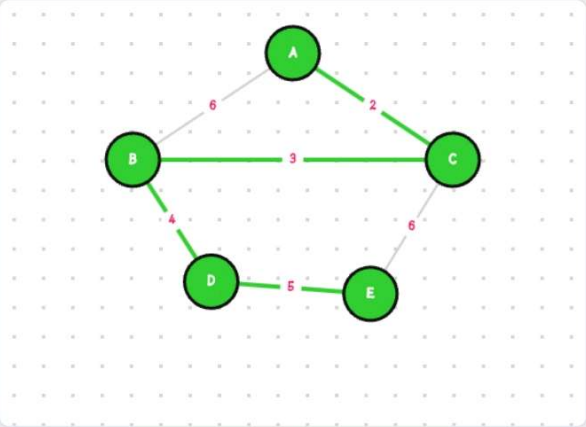
Start Node: A Speed:

BFS | DFS | Dijkstra | Floyd-Warshall | Prim's (MST) | Kruskal's (MST)

Reset View

Sorted Edge List

C-E(6)



Default | Source (i) | Dest (j) | Via (k)

Kruskal's Algorithm (MST)

Sorts ALL edges globally and uses Disjoint Set (Union-Find).

Time: O(V + E)
Space: O(V)

Execution Log

```
MST!
[18:56:25] Evaluating edge D-E (Wt: 5)...
[18:56:26] ↳ Include: Added to MST!
[18:56:26] Evaluating edge A-B (Wt: 6)...
[18:56:27] ↳ Discard: Creates a cycle.
[18:56:27] Evaluating edge C-E (Wt: 6)...
[18:56:28] ↳ Discard: Creates a cycle.
[18:56:28] Finished: MST built! Weight: 14
```

Kruskal's Minimum Spanning Tree

Total Weight: 14

A ↔ C (Wt: 2) | B ↔ C (Wt: 3) | B ↔ D (Wt: 4) | D ↔ E (Wt: 5)

6:56 Vo) LTE 0.80 KB/s 4G+ 48%

ukka-venugopalam.github.io 13

Interactive Graph Algorithm Visualizer

Run | Edit | Delete | Clear Canvas

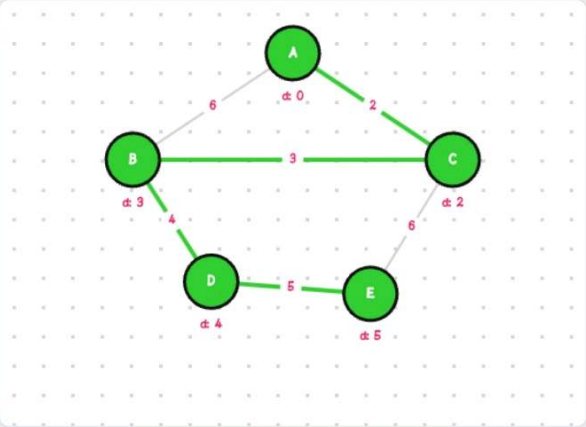
Start Node: A Speed:

BFS | DFS | Dijkstra | Floyd-Warshall | Prim's (MST) | Kruskal's (MST)

Reset View

Priority Queue

B(6) E(6)



Default | Source (i) | Dest (j) | Via (k)

Prim's Algorithm (MST)

Builds MST by picking cheapest edges. Uses Priority Queue.

```
Time: O(V + E)
Space: O(V)
```

Execution Log

```
[18:56:12] ↳ Update: Cheaper edge to B (Wt: 6).
[18:56:12] ↳ Update: Cheaper edge to C (Wt: 2).
[18:56:14] ↳ Update: Cheaper edge to B (Wt: 3).
[18:56:14] ↳ Update: Cheaper edge to E (Wt: 6).
[18:56:15] ↳ Update: Cheaper edge to D (Wt: 4).
[18:56:17] ↳ Update: Cheaper edge to E (Wt: 5).
[18:56:18] Finished: MST built! Weight: 14
```

Prim's Minimum Spanning Tree

Total Weight: 14

A ↔ C (Wt: 2) | C ↔ B (Wt: 3) | B ↔ D (Wt: 4) | D ↔ E (Wt: 5)